Project Documentation

Project Title

BioLink: Fingerprint-Based Family Identification System

Objective

- Register fingerprints with personal and family details.
- Identify individuals by scanning fingerprints.
- Retrieve family information: parents, siblings, and contact numbers.
- Build a functional genealogy system for law enforcement or genealogy research.

Scope

- Prototype web application with **fingerprint registration, matching, and family retrieval**.
- Scalable for AI-based improvements in fingerprint matching and kinship prediction.
- Secure handling of sensitive personal and biometric data.

Technology Stack

Layer	Technology
Frontend	Angular / React
Backend	Node.js + Express.js
Database	MongoDB
Fingerprint Matching	SourceAFIS (Java/Python) or AI CNN service
Authentication & Security	JWT, bcrypt, HTTPS
Optional AI	Kinship prediction, Face recognition, Match ranking

Features

1. **User Registration**
   - Register name, age, phone number, father and mother details.

- Capture fingerprint and convert to a template.
- Store family relationships dynamically in the database.

2. **Fingerprint Matching**
   - Scan fingerprint to identify a user.
   - Return match percentage and user info.
   - Works even with partial or low-quality fingerprints (AI integration optional).
3. **Family Retrieval**
   - Fetch parents, siblings, and children from the database.
   - Build a simple family tree.
4. **Admin Dashboard**
   - Manage users and family relationships.
   - Verify fingerprint scans and update data.
5. **Security & Privacy**
   - Fingerprints stored as templates, not raw images.
   - Encryption for sensitive data (phone numbers, personal info).
   - Secure API endpoints with JWT and HTTPS.

Database Design

Users Collection

```
{
 "_id": "uid004",
 "name": "Aarav Sharma",
 "age": 12,
 "phone": "9876543210",
 "photo": "photo_url",
 "fingerprint_template": "encoded_data",
 "family": [
  { "relation": "father", "user_id": "uid001" },
  { "relation": "mother", "user_id": "uid002" }
 ]
}
```

Parent Example

```
{
```

```
  "_id": "uid001",
  "name": "Rohit Sharma",
  "age": 40,
  "fingerprint_template": "encoded_data",
  "family": [
    { "relation": "child", "user_id": "uid004" }
  ]
}
{
  "_id": "uid002",
  "name": "Priya Sharma",
  "age": 38,
  "fingerprint_template": "encoded_data",
  "family": [
    { "relation": "child", "user_id": "uid004" }
  ]
}
```


Workflow

1. Registration

1. User fills in form (name, age, phone, parents).
2. Fingerprint scanned and converted to a template.
3. Data stored in **MongoDB**, family relationships stored via user_id references.

2. Fingerprint Matching

1. User scans fingerprint on portal.
2. Backend compares fingerprint template with DB (SourceAFIS/AI).
3. Returns **best match ID + match percentage**.

3. Family Retrieval

1. Backend fetches family array of matched user.
2. Retrieves parents, siblings, and children.
3. Returns **family tree + contact info** to frontend.

API Endpoints

Endpoint	Method	Description
/api/auth/register	POST	Register user with fingerprint and family info
/api/auth/login	POST	User login
/api/fingerprint/match	POST	Match scanned fingerprint
/api/family/:id	GET	Fetch family details for user ID
/api/result/:id	GET	View match result + family

AI Integration (Optional / Future)

- **Fingerprint Recognition**: CNN-based models for partial or poor-quality prints.
- **Kinship Prediction**: Siamese networks for estimating relationship probability.
- **Face Recognition**: Cross-verify identity with photo.
- **Match Ranking**: Return top probable matches instead of a single match.

Security Considerations

- Fingerprint templates stored instead of raw images.
- Personal and contact info encrypted.
- HTTPS for secure data transfer.
- JWT for authentication on sensitive endpoints.

Future Enhancements

- Live fingerprint scanning using biometric SDK.
- Multi-level family tree visualization (grandparents, cousins).
- Integration with official ID databases (Aadhaar, police records).
- AI-driven automatic kinship prediction for missing child identification.

Conclusion

This project builds a **secure and functional fingerprint-based identification and genealogy system**.

- Identifies individuals via fingerprints.
- Retrieves family details dynamically.
- Can be extended with **AI** and official database integration for real-world applications.

✅ This version covers **everything important**: registration, database, family relationships, fingerprint matching, API structure, workflow, AI, security, and future scope.

If you want, I can **also make a small visual diagram** showing **fingerprint scan → backend → family retrieval → frontend**, which will make the documentation **very readable and professional**.

Do you want me to make that diagram?

Step-by-Step Project Plan: BioLink Fingerprint App

Phase 0: Preparation (1–2 Days)

1. **Decide Tech Stack**
   - Frontend: **Angular or React**
   - Backend: **Node.js + Express.js**
   - Database: **MongoDB**
   - Fingerprint: **SourceAFIS SDK** (Java/Python) or buy a USB fingerprint scanner
2. **Install Dev Tools**
   - Node.js + npm
   - MongoDB (local or cloud like MongoDB Atlas)
   - Angular/React CLI
   - VS Code or preferred IDE

Phase 1: Fingerprint Hardware & SDK (2–3 Days)

1. **Buy Fingerprint Scanner**
   - Budget: **₹2,000–₹5,000** for USB scanner (e.g., Digital Persona U.are.U 4500)
2. **Install SDK / Drivers**
   - Usually comes with the scanner
   - Test scanning → make sure you can **capture fingerprint as an image or template**
3. **Test Capture Script**

- Simple Python/JS file to **capture and save fingerprint template**

Example (Node.js / Express)

- File: fingerprintCapture.js

```
// Pseudo code: captures fingerprint and saves template

const captureFingerprint = () => {

  const template = scanner.capture(); // scanner SDK method

  console.log('Fingerprint captured:', template);

  return template;

};
```

- Run this **first** to ensure your scanner works.

Phase 2: Backend Setup (3–4 Days)

1. **Create Express.js Project**
2. mkdir BioLinkBackend
3. cd BioLinkBackend
4. npm init -y
5. npm install express mongoose body-parser cors

Create Core Backend Files

BioLinkBackend/

```
├── config/
│   └── db.js                          # MongoDB connection
├── controllers/
│   └── authController.js              # Handles registration/login
│   └── fingerprintController.js       # Handles fingerprint matching
├── models/
│   └── User.js                        # MongoDB schema for users
```

```
├── routes/
│   └── auth.js                    # API routes for registration/login
│   └── fingerprint.js             # API routes for fingerprint scan/match
├── services/
│   └── fingerprintService.js      # Functions to compare fingerprints
├── utils/
│   └── encrypt.js                 # Optional: encrypt/decrypt data
├── index.js                       # Main server file
├── package.json
└── .env                           # Environment variables (DB URL, secrets)
```

Create Core Frontend Files

BioLinkFrontend/

```
├── src/
│   ├── app/
│   │   ├── components/
│   │   │   ├── register/
│   │   │   │   ├── register.component.ts
│   │   │   │   ├── register.component.html
│   │   │   │   └── register.component.css
│   │   │   ├── scan-fingerprint/
│   │   │   │   ├── scan-fingerprint.component.ts
│   │   │   │   ├── scan-fingerprint.component.html
│   │   │   │   └── scan-fingerprint.component.css
│   │   │   ├── family-tree/
```

│ │ │ │ ├── family-tree.component.ts

│ │ │ │ ├── family-tree.component.html

│ │ │ │ └── family-tree.component.css

│ │ │ └── result/

│ │ │ ├── result.component.ts

│ │ │ ├── result.component.html

│ │ │ └── result.component.css

│ │ ├── services/

│ │ │ └── api.service.ts

│ │ ├── app-routing.module.ts

│ │ └── app.module.ts

│ ├── assets/

│ │ └── images/ # Optional: photos for users

│ ├── environments/

│ │ ├── environment.ts # API URLs and configs

│ │ └── environment.prod.ts

│ └── index.html

├── angular.json

├── package.json

└── tsconfig.json

1. **Write DB Connection (config/db.js)**

import mongoose from 'mongoose';

mongoose.connect('mongodb://localhost:27017/biolink', { useNewUrlParser: true, useUnifiedTopology: true })

```
.then(()=> console.log('DB Connected'))

.catch(err => console.error(err));
```

1. **Create User Schema (models/User.js)**

```
import mongoose from 'mongoose';

const userSchema = new mongoose.Schema({
  name: String,
  age: Number,
  phone: String,
  fingerprint_template: String,
  family: [{ relation: String, user_id: String }]
});

export default mongoose.model('User', userSchema);
```

Phase 3: User Registration API (2–3 Days)

- File: routes/auth.js
- Create POST /register endpoint:

```
router.post('/register', async (req, res) => {
  const { name, age, phone, fingerprint_template, family } = req.body;
  const user = new User({ name, age, phone, fingerprint_template, family });
  await user.save();
  res.json({ message: 'User registered successfully' });
});
```

- Test with **Postman** before integrating frontend.

Phase 4: Fingerprint Matching API (2–3 Days)

- File: routes/fingerprint.js

```
router.post('/match', async (req, res) => {
  const { fingerprint_template } = req.body;
```

```
  const users = await User.find();
  let bestMatch = null;
  let maxScore = 0;

  users.forEach(u => {
    const score = compareFingerprints(fingerprint_template, u.fingerprint_template);
    if(score > maxScore){ maxScore = score; bestMatch = u; }
  });

  if(bestMatch) res.json({ user: bestMatch, matchPercent: maxScore });
  else res.json({ message: 'No match found' });
});
```

- compareFingerprints → use **SourceAFIS** or your scanner SDK method.

Phase 5: Frontend Setup (3–4 Days)

1. Create Angular/React project.
2. Create pages:
   - **Register** → form with name, phone, family, fingerprint scan.
   - **Scan** → capture fingerprint → POST /match.
   - **Result** → show match + family tree.
3. Test API integration with **dummy data first**, then with real scanner.

Phase 6: Family Tree / Data Retrieval (2–3 Days)

- Fetch family details using family array in matched user.
- Optional: Use **D3.js / vis.js** for visual tree.

Phase 7: Testing & Security (3 Days)

- Test registration, matching, family retrieval.
- Encrypt fingerprints and phone numbers.
- Use HTTPS if deployed.

Phase 8: Deployment

- Deploy backend (Heroku, Railway, or AWS EC2).

- Deploy frontend (Vercel / Netlify).
- Test scanner integration in live environment.

Timeline Summary

Phase	Days
Preparation	1–2
Fingerprint Scanner Test	2–3
Backend Setup	3–4
Registration API	2–3
Matching API	2–3
Frontend Setup	3–4
Family Tree	2–3
Testing & Security	3
Deployment	1–2
Total: ~20–25 days	

💡 Pro Tips

1. **Start with backend first** → APIs for registration + matching.
2. **Test with dummy fingerprint templates** before buying scanner.
3. **Integrate scanner last** → once APIs and DB work.
4. **Keep family references simple** → father + mother + children for MVP.
5. Add AI or advanced features **only after core functionality works**.

If you want, I can make a **ready-to-use folder/file structure with example code for backend + fingerprint capture + MongoDB**, so you can literally **start coding tomorrow** without confusion.

Do you want me to do that next?

You said:

what about db?

ChatGPT said:

Ah, yes! Let's focus fully on the **database part**, Mayank. Since this project revolves around fingerprints **and family relationships**, the database is **critical**. I'll explain **how to design, structure, and connect it step by step**.

Database Choice

- **MongoDB** is perfect here because:
  - Flexible schema → family relationships can grow dynamically.
  - Easy to store JSON-like objects (good for family arrays).
  - Works well with **Node.js + Express**.
- You can start with:
  - **Local MongoDB** during development
  - **MongoDB Atlas** for cloud deployment

Database Design

Users Collection

Each user stores:

- Personal info (name, age, phone, photo)
- Fingerprint template (encoded, not raw image)
- Family references (father, mother, siblings, children)

Example:


```
{
  "_id": "uid004",
  "name": "Aarav Sharma",
  "age": 12,
  "phone": "9876543210",
  "photo": "photo_url",
  "fingerprint_template": "encoded_template_here",
  "family": [
    { "relation": "father", "user_id": "uid001" },
    { "relation": "mother", "user_id": "uid002" },
```

{ "relation": "sibling", "user_id": "uid005" }

]

}

Parent Example:

{

"_id": "uid001",

"name": "Rohit Sharma",

"age": 40,

"phone": "9876501234",

"fingerprint_template": "encoded_template_here",

"family": [

{ "relation": "child", "user_id": "uid004" },

{ "relation": "child", "user_id": "uid005" }

]

}

Why Not Separate Collections for Family?

- You could have a families collection, but keeping family references **inside user document** is simpler for MVP.
- Easy to query:
  - Get all children of a parent → search family array.
  - Get parents of a child → read family array of child.

Connecting DB with Backend (Express.js)

- Install mongoose:

npm install mongoose

- config/db.js

import mongoose from 'mongoose';

const connectDB = async () => {

```
  try {
    await mongoose.connect('mongodb://localhost:27017/biolink', {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });
    console.log('MongoDB Connected');
  } catch (err) {
    console.error('DB connection failed', err);
    process.exit(1);
  }
};

export default connectDB;
```

- Connect in index.js:

```
import express from 'express';
import connectDB from './config/db.js';

const app = express();
connectDB();

app.use(express.json());

// routes here
```

Sample Queries

Add a new user

```
const user = new User({
```

```
  name: "Aarav Sharma",
  age: 12,
  phone: "9876543210",
  fingerprint_template: "encoded_data",
  family: [
    { relation: "father", user_id: "uid001" },
    { relation: "mother", user_id: "uid002" }
  ]
});
await user.save();
```

Get family of a user

```
const user = await User.findById("uid004");
const familyDetails = await Promise.all(
  user.family.map(async member => {
    const relative = await User.findById(member.user_id);
    return { relation: member.relation, name: relative.name };
  })
);
console.log(familyDetails);
```

Match fingerprint

```
const allUsers = await User.find();
allUsers.forEach(u => {
   const score = compareFingerprints(scannedTemplate, u.fingerprint_template);
   if(score > bestScore){ bestScore = score; bestMatch = u; }
});
```

Notes

1. **Store fingerprint as encoded template**, not raw image.
2. **Family relationships stored by user_id** for easy linking.
3. **Use MongoDB indexes** on fingerprint field if DB grows large.
4. Optional: For production, encrypt sensitive fields (phone, fingerprint template).

💡 **Summary**

- MongoDB stores **all users and their family references**.
- Fingerprint template is **unique key** → allows scanning → match → retrieve family.
- Queries are **simple**, dynamic, and scalable.

If you want, I can **draw a full DB schema diagram** showing **users, family relations, and fingerprint connections** — this makes the structure **super clear and visual** for your project.

Full File Structure 😀

BioLinkProject/

├── backend/ # Node.js + Express backend

│ ├── config/

│ │ └── db.js # MongoDB connection

│ ├── controllers/

│ │ ├── authController.js # Handles registration/login

│ │ └── fingerprintController.js # Handles fingerprint matching

│ ├── models/

│ │ └── User.js # MongoDB schema for users

│ ├── routes/

│ │ ├── auth.js # Auth routes: register/login

│ │ └── fingerprint.js # Fingerprint match routes

```
│   ├── services/
│   │   └── fingerprintService.js # Compare fingerprint templates
│   ├── utils/
│   │   └── encrypt.js       # Encrypt/decrypt sensitive data
│   ├── index.js             # Main server file
│   ├── package.json
│   └── .env                 # DB URL, secret keys
│
├── frontend/                # Angular frontend
│   ├── angular.json
│   ├── package.json
│   ├── tsconfig.json
│   ├── src/
│   │   ├── index.html       # Main HTML
│   │   ├── main.ts          # Angular entry point
│   │   ├── app/
│   │   │   ├── app.module.ts
│   │   │   ├── app-routing.module.ts
│   │   │   ├── components/
│   │   │   │   ├── register/
│   │   │   │   │   ├── register.component.ts
│   │   │   │   │   ├── register.component.html
│   │   │   │   │   └── register.component.css
│   │   │   │   ├── scan-fingerprint/
│   │   │   │   │   ├── scan-fingerprint.component.ts
│   │   │   │   │   ├── scan-fingerprint.component.html
```

```
│   │   │   │   │   └── scan-fingerprint.component.css
│   │   │   │   ├── family-tree/
│   │   │   │   │   ├── family-tree.component.ts
│   │   │   │   │   ├── family-tree.component.html
│   │   │   │   │   └── family-tree.component.css
│   │   │   │   └── result/
│   │   │   │       ├── result.component.ts
│   │   │   │       ├── result.component.html
│   │   │   │       └── result.component.css
│   │   │   ├── services/
│   │   │   │   └── api.service.ts  # Calls backend APIs
│   │   │   └── models/
│   │   │       └── user.model.ts   # Optional TS interface for User
│   │   ├── assets/
│   │   │   └── images/      # User photos, icons
│   │   └── environments/
│   │       ├── environment.ts      # API URL and configs
│   │       └── environment.prod.ts
│
└── README.md             # Project documentation
```